

Foundations of 4D/6D Object Capture for Virtual Environments

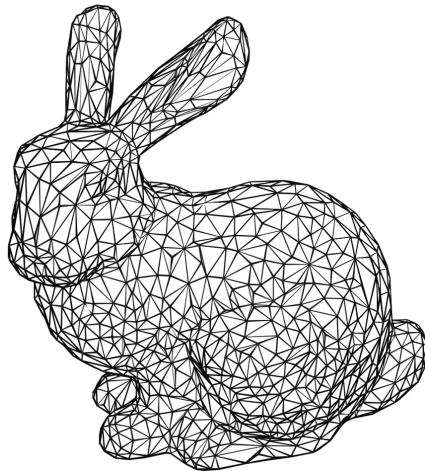
Reconstruction Methods

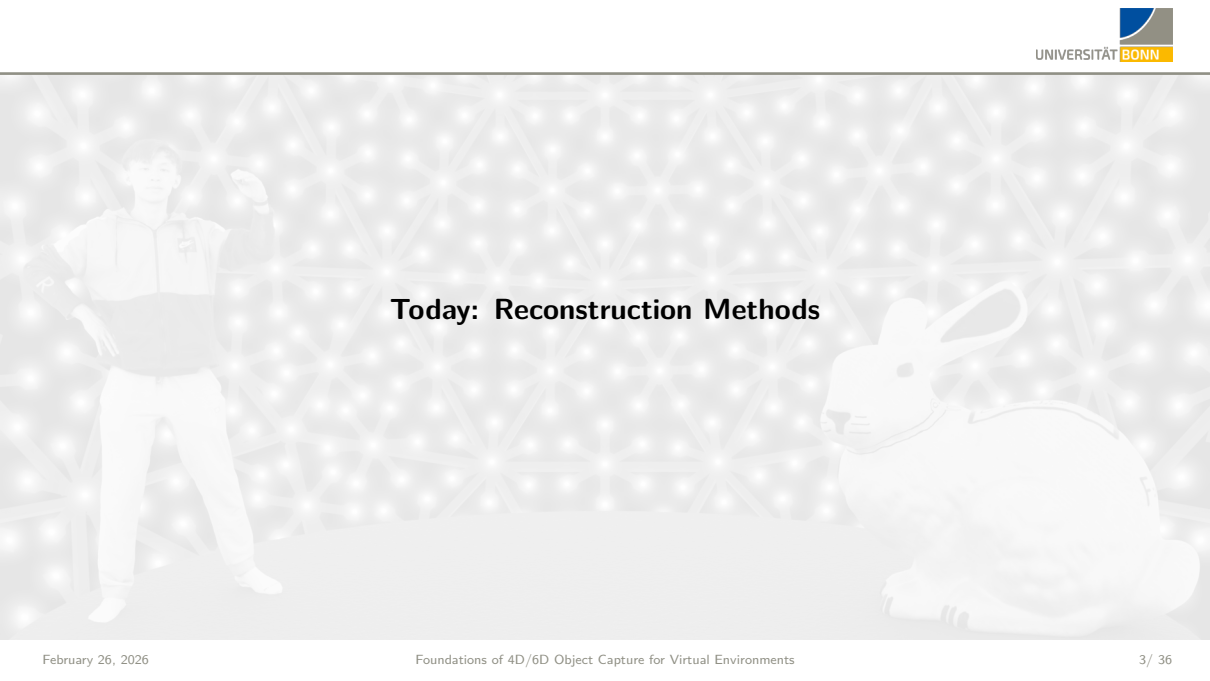


February 26, 2026



→ **Scene Representations**



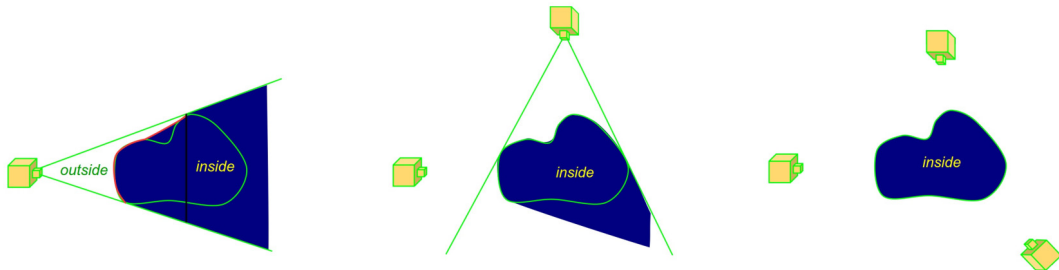


Today: Reconstruction Methods

Scenario: Capture an object from different angles, so we obtain a sequence of depth maps

$$D_1, \dots, D_n$$

Problem: How to reconstruct a surface from *all* measurements?



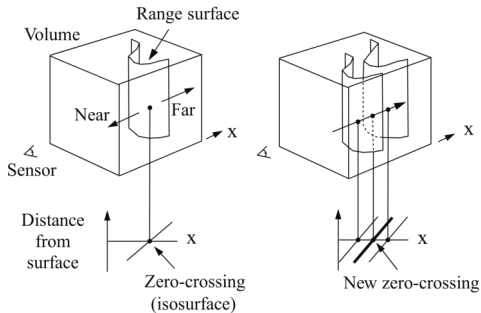
Idea: Fuse all measurements into a single volume¹

1. Compute signed distance functions sdf_i from depth maps D_i
2. Use weight functions w_i to account for sensor noise
3. Merge into weighted average:

$$\text{SDF}(\mathbf{x}) = \frac{\sum_{i=1}^n w_i(\mathbf{x}) \text{sdf}_i(\mathbf{x})}{\sum_{i=1}^n w_i(\mathbf{x})}$$

Problem: How to compute average if only current/last depth map is known?

→ Online update formula



¹B. Curless and M. Levoy. "A Volumetric Method for Building Complex Models from Range Images". In: *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques. SIGGRAPH '96*. ACM, 1996.

Problem: How to compute average if only current depth map is known?

Lemma (Weighted Cumulative Moving Average)

Let $\text{sdf}_1(\mathbf{x}), \dots, \text{sdf}_{n+1}(\mathbf{x})$ be a sequence of signed distance functions from some measurements and let $w_1(\mathbf{x}), \dots, w_{n+1}(\mathbf{x})$ be a sequence of weights. Then the weighted average

$$\text{SDF}_{n+1}(\mathbf{x}) = \frac{\sum_{i=1}^{n+1} w_i(\mathbf{x}) \text{sdf}_i(\mathbf{x})}{\sum_{i=1}^{n+1} w_i(\mathbf{x})}$$

can be computed online using the formula

$$\begin{aligned}\text{SDF}_{n+1}(\mathbf{x}) &= \frac{W_n(\mathbf{x}) \text{SDF}_n(\mathbf{x}) + w_{n+1}(\mathbf{x}) \text{sdf}_{n+1}(\mathbf{x})}{W_n(\mathbf{x}) + w_{n+1}(\mathbf{x})} \\ W_{n+1}(\mathbf{x}) &= W_n(\mathbf{x}) + w_{n+1}(\mathbf{x})\end{aligned}$$

→ Only need to store **2 values** to compute the weighted average!

Proof:

The current weighted distance can be rewritten as

$$\begin{aligned}w_{n+1}(\mathbf{x}) \text{sdf}_{n+1}(\mathbf{x}) &= \sum_{i=1}^{n+1} w_i(\mathbf{x}) \text{sdf}_i(\mathbf{x}) - \sum_{i=1}^n w_i(\mathbf{x}) \text{sdf}_i(\mathbf{x}) \\&= \left(\sum_{i=1}^{n+1} w_i(\mathbf{x}) \right) \text{SDF}_{n+1}(\mathbf{x}) - \left(\sum_{i=1}^n w_i(\mathbf{x}) \right) \text{SDF}_n(\mathbf{x}) \\&= W_{n+1}(\mathbf{x}) \text{SDF}_{n+1}(\mathbf{x}) - W_n(\mathbf{x}) \text{SDF}_n(\mathbf{x})\end{aligned}$$

Rearranging the terms and using $W_{n+1}(\mathbf{x}) = W_n(\mathbf{x}) + w_{n+1}(\mathbf{x})$ leads to

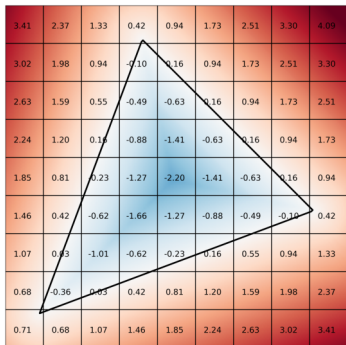
$$\text{SDF}_{n+1}(\mathbf{x}) = \frac{W_n(\mathbf{x}) \text{SDF}_n(\mathbf{x}) + w_{n+1}(\mathbf{x}) \text{sdf}_{n+1}(\mathbf{x})}{W_n(\mathbf{x}) + w_{n+1}(\mathbf{x})}$$

Observation: Very large distances to surface not important $\rightarrow$ only small band, i.e. truncation region μ

$$\text{tsdf}(\mathbf{p}) = \begin{cases} \mu & \text{if } \text{sdf}(\mathbf{p}) > \mu \\ \text{sdf}(\mathbf{p}) & \text{if } |\text{sdf}(\mathbf{p})| \leq \mu \\ \text{N/A} & \text{if } \text{sdf}(\mathbf{p}) < -\mu \end{cases}$$

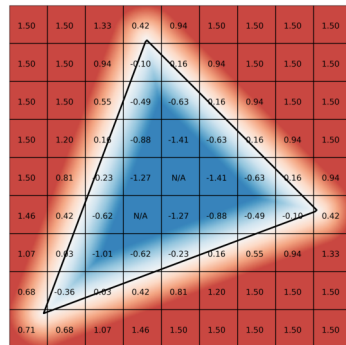
[empty space to camera]

[unknown scene behind]



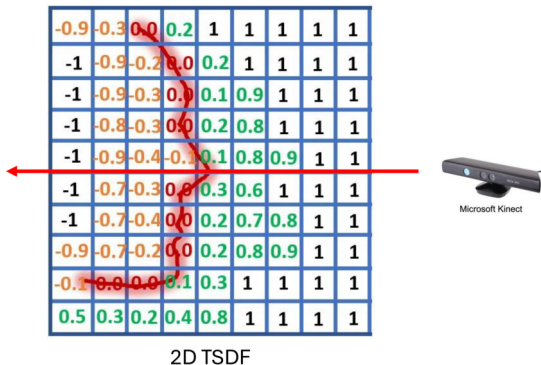
Example:

$$\mu = 1.5$$



(Projective) Truncated Signed Distance Fields

Back to volumetric fusion, how can we compute the (truncated) signed distance function from depth maps D_i ?



1. Transform point p to local camera coordinates

$$p_{\text{cam}} = [R | t] \cdot p$$

2. Project local point into depth map and compute SDF as depth difference

$$\text{sdf}_i(p) = \underbrace{D_i(\pi(K \cdot p_{\text{cam}}))}_{\text{measured depth}} - \underbrace{z(p_{\text{cam}})}_{\text{point depth}}$$

3. Truncate SDF

Caution: This *projective SDF* is **not** a "true" signed distance function!

$$\text{sdf}_i(\mathbf{p}) = \underbrace{D_i(\pi(\mathbf{K} \cdot \mathbf{p}_{\text{cam}}))}_{\text{measured depth}} - \underbrace{z(\mathbf{p}_{\text{cam}})}_{\text{point depth}}$$



Properties:

- ▶ Only correct at the surface level $\text{sdf}_i(\mathbf{x}) = 0$
- ▶ Accuracy depends on the geometry of the surface and may vary drastically (e.g. for grazing angles)
- ▶ Error reduces when fusing more views from different angles

Results: Stanford 3D Scanning Repository

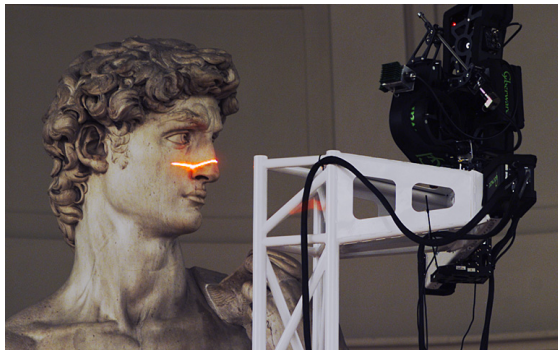


10 scans, 35947 vertices, 69451 triangles



114 scans, 345944 triangles

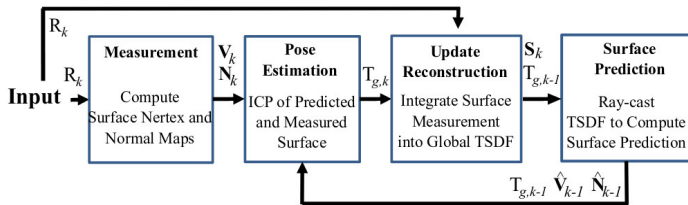
Results: Digital Michelangelo Project²



- Mesh with 8 million polygons reconstructed from 2.0 mm laser scans (remarkable at that time!)
- Later in 2009 scanned at 0.25 mm resolution resulting in around 1 billion polygons

²M. Levoy et al. "The Digital Michelangelo Project: 3D Scanning of Large Statues". In: *Annual Conference on Computer Graphics and Interactive Techniques*. 2000.

First system for scene reconstruction with single RGB-D camera *in real time* fully on a GPU³⁴



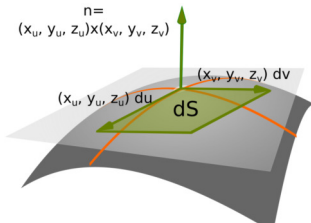
³S. Izadi et al. "KinectFusion: Real-time 3D Reconstruction and Interaction Using a Moving Depth Camera". In: *ACM Symposium on User Interface Software and Technology (UIST)*. 2011.

⁴R. A. Newcombe et al. "KinectFusion: Real-Time Dense Surface Mapping and Tracking". In: *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 2011.

First, perform pre-processing of the raw RGB-D data

- ▶ (Optional) Compute vertex map V_i by unprojecting depth map D_i
→ Mainly for convenience in later steps
- ▶ Compute normal map from vertex map V_i / depth map D_i (forward differences)

$$N_i(u, v) = \frac{(\mathbf{V}_i(u+1, v) - \mathbf{V}_i(u, v)) \times (\mathbf{V}_i(u, v+1) - \mathbf{V}_i(u, v))}{\|(\mathbf{V}_i(u+1, v) - \mathbf{V}_i(u, v)) \times (\mathbf{V}_i(u, v+1) - \mathbf{V}_i(u, v))\|}$$



→ To reduce approximation error $O(h)$, we could also use, e.g., central differences ($O(h^2)$)

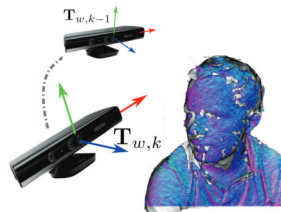
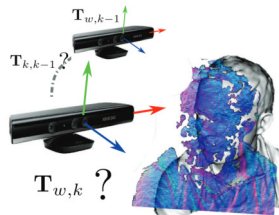
Task: Find poses/transformations of camera for each recorded RGB-D image

→ Hierarchical Point-to-Plane Iterative Closest Point (ICP)

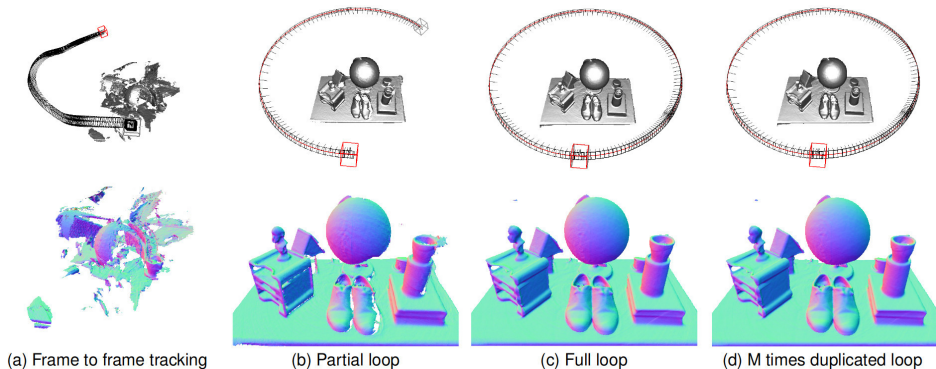
Idea: Iterate between the following steps:

1. Select source points
2. Find matching points in target scan
3. Weight correspondences according to confidence
4. **Reject outlier pairs** → **important**
5. Minimize (point-to-plane) error function

After some iterations, go to next of 3 resolution levels and repeat

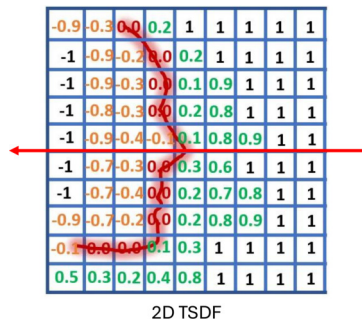
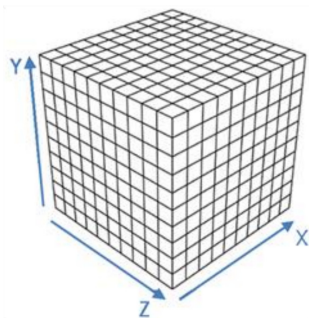


Question: Which "target" should be chosen?



- Frame-to-Frame tracking accumulates drift resulting in a broken reconstruction
- Frame-to-Model tracking is significantly less susceptible to drift

Volumetric Integration by Curless and Levoy (1996)

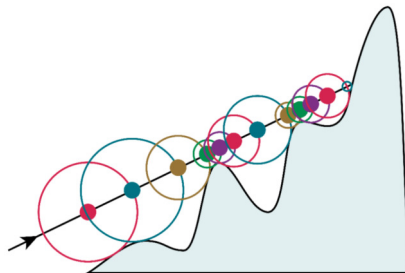


- Online update formula for the *projective TSDF*:

$$\text{TSDF}_{n+1}(\mathbf{x}) = \frac{W_n(\mathbf{x}) \text{TSDF}_n(\mathbf{x}) + w_{n+1}(\mathbf{x}) \text{tsdf}_{n+1}(\mathbf{x})}{W_n(\mathbf{x}) + w_{n+1}(\mathbf{x})}$$

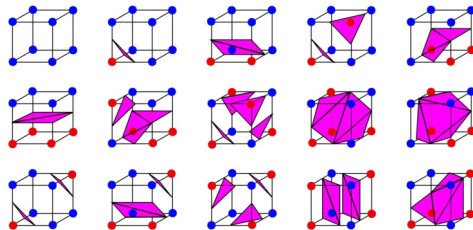
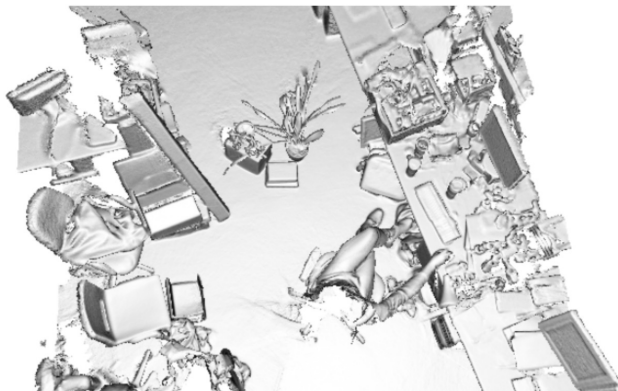
$$W_{n+1}(\mathbf{x}) = W_n(\mathbf{x}) + w_{n+1}(\mathbf{x})$$

Predict current state of reconstruction by rendering the TSDF



- ▶ Can use Sphere Tracing to accelerate ray marching
- ▶ Jumping behind the surface possible since TSDF is (projective) approximation
→ Sign flip of SDF value can detect this, so automatically marching back here
- ▶ Predicted surface used in next frame for pose estimation

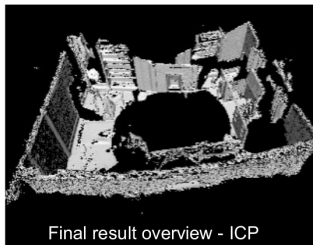
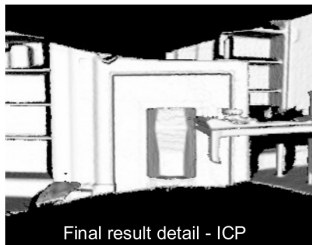
After the live reconstruction session, a mesh of the scene can be extracted



→ Marching Cubes

Limitations:

- ▶ High memory consumption: $512 \times 512 \times 512$ voxel grid in GPU memory
 - Does not scale to larger scenes
- ▶ Frame-to-Model camera tracking
 - Requires strong geometric features in the scene
 - No explicit loop closure for globally consistent reconstruction



→ No recovery from tracking losses

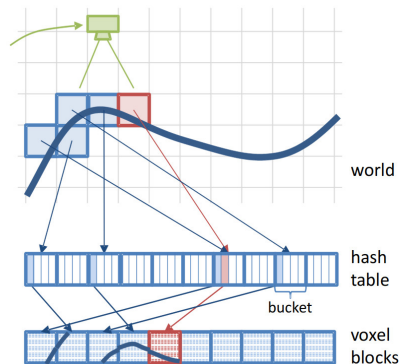
Idea: Sparse representation of the scene⁵

- ▶ Summarize $8^3 = 512$ voxel into a voxel block
- ▶ Only store voxel blocks inside the truncation region μ
- ▶ Manage unordered set of voxel blocks with spatial hashing

$$H(x, y, z) = (x \cdot p_1 \oplus y \cdot p_2 \oplus z \cdot p_3) \bmod n$$

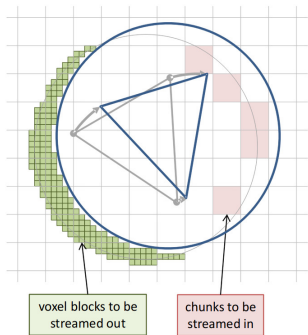
where:

- ▶ $(x, y, z) \in \mathbb{Z}^3$ are discrete voxel block coordinates
- ▶ $p_1 = 73856093$, $p_2 = 19349669$, $p_3 = 83492791$ are large prime numbers
- ▶ n is the number of buckets in the hash table
- ▶ $\oplus$ is the XOR operator



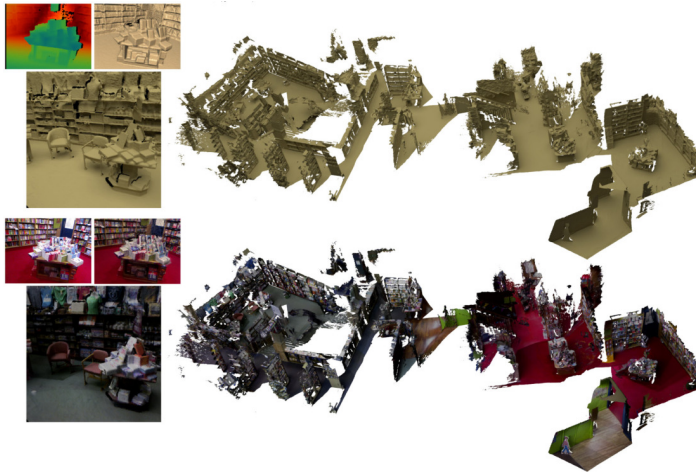
⁵M. Nießner et al. "Real-time 3D Reconstruction at Scale using Voxel Hashing". In: *ACM Transactions on Graphics (TOG)* (2013).

Idea: Sparse representation of the scene

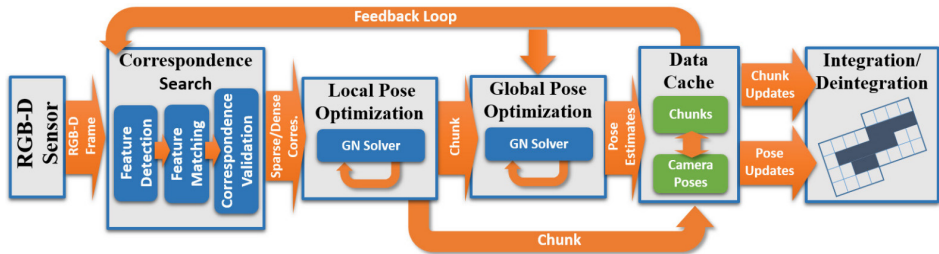


- Stream voxel blocks between CPU and GPU memory
 - Allows to reconstruct much larger scenes
 - Voxel blocks are summarized as chunks on the CPU and managed using a simple list

Results:



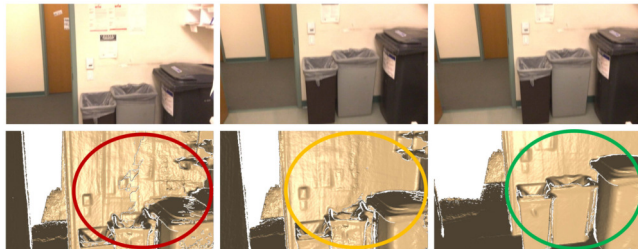
Idea: Online global optimization of camera poses⁶



- ▶ Detect sparse SIFT features in RGB-D images
- ▶ Use chunks of 10 RGB-D images and perform local bundle adjustment of the SIFT features
- ▶ Perform global bundle adjustment of the chunks for consistent camera pose tracking
- ▶ Incrementally integrate and deintegrate the chunks into the VoxelHashing-like scene representation

⁶A. Dai et al. "BundleFusion: Real-time Globally Consistent 3D Reconstruction using On-the-fly Surface Re-integration". In: *ACM Transactions on Graphics 2017 (TOG)* (2017).

Results: Re-integration after loop closure



- **Integration** and **Re-integration**: Common online update formula when integrating i -th image:

$$\text{TSDF}_{\text{new}}(\mathbf{x}) = \frac{W_{\text{old}}(\mathbf{x}) \text{TSDF}_{\text{old}}(\mathbf{x}) + w_i(\mathbf{x}) \text{tsdf}_i(\mathbf{x})}{W_{\text{old}}(\mathbf{x}) + w_i(\mathbf{x})}$$

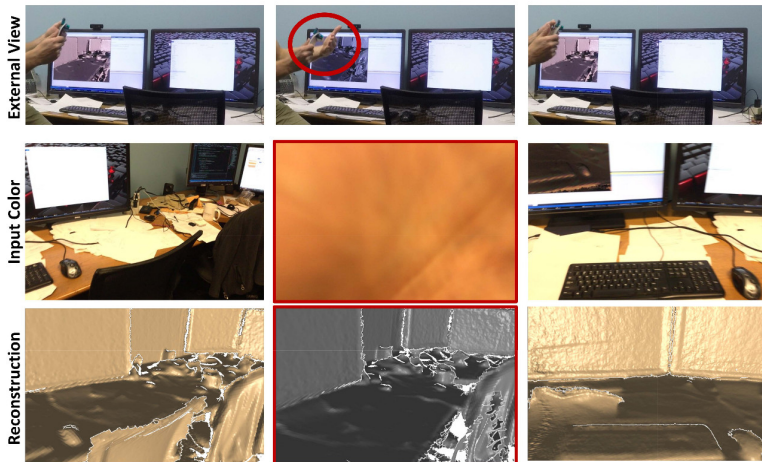
$$W_{\text{new}}(\mathbf{x}) = W_{\text{old}}(\mathbf{x}) + w_i(\mathbf{x})$$

- **De-integration**: Can revert this process

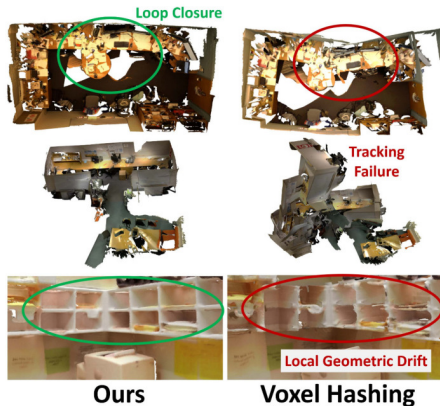
$$\text{TSDF}_{\text{old}}(\mathbf{x}) = \frac{W_{\text{new}}(\mathbf{x}) \text{TSDF}_{\text{new}}(\mathbf{x}) - w_i(\mathbf{x}) \text{tsdf}_i(\mathbf{x})}{W_{\text{new}}(\mathbf{x}) - w_i(\mathbf{x})}$$

$$W_{\text{old}}(\mathbf{x}) = W_{\text{new}}(\mathbf{x}) - w_i(\mathbf{x})$$

Results: Relocalization after tracking loss

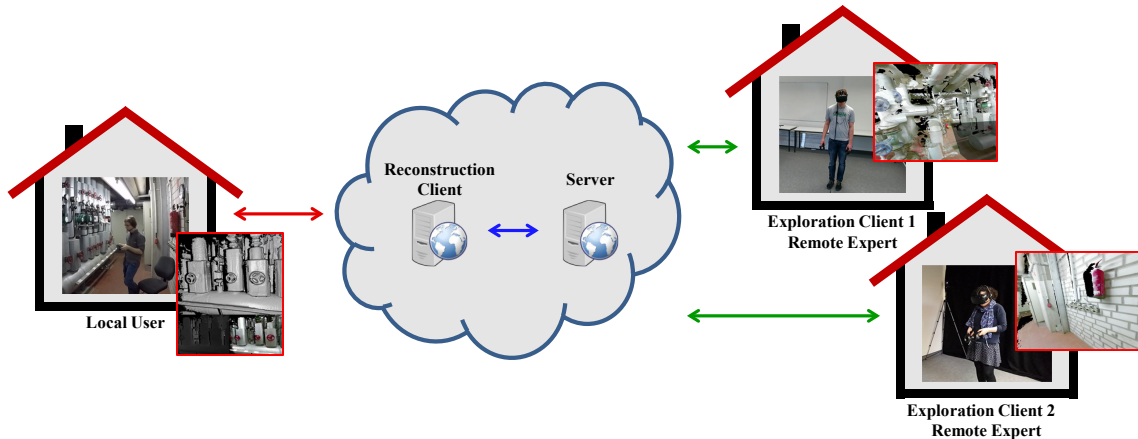


Results: Globally consistent 3D reconstructions

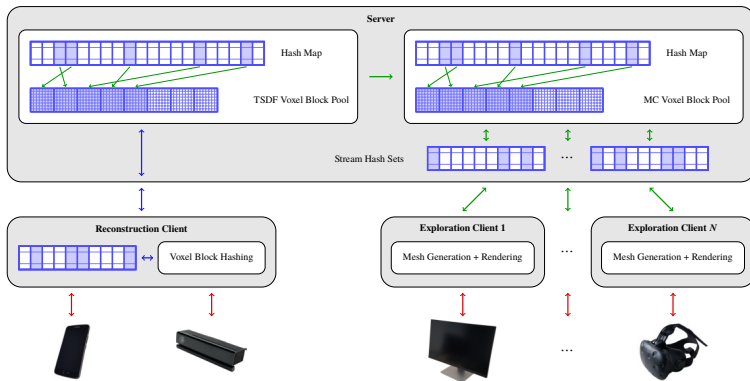


<https://youtu.be/keIirXrRb1k?t=8>

Real-time telepresence system for streaming *places* to multiple users⁷



⁷P. Stotko et al. "SLAMCast: Large-Scale, Real-Time 3D Reconstruction and Streaming for Immersive Multi-Client Live Telepresence". In: *IEEE Transactions on Visualization and Computer Graphics* 25.5 (2019).



- **Reconstruction Client:** Performs reconstruction via VoxelHashing and streams TSDF data to server
- **Server:** Compresses TSDF voxel to Marching Cubes voxel
→ 12 Bytes (TSDF, Fusion Weight, Color, Padding) down to 4 Byte (MC Index, Color)
- **Exploration Client:** Finalize Marching Cubes step and render scene on VR headset

Results:



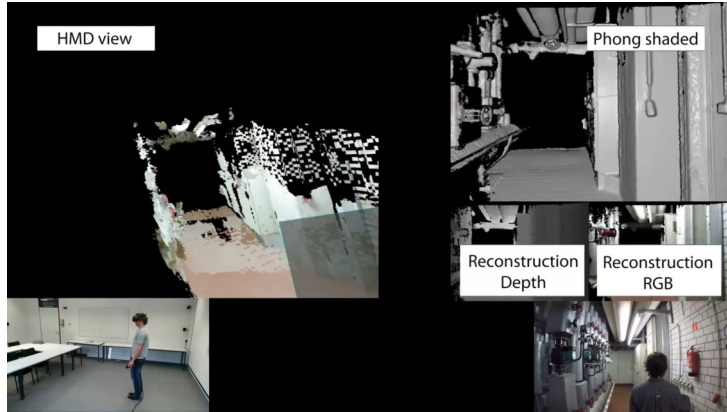
(a) With Color Interpolation.



(b) Without Color Interpolation.

- ▶ Quality slightly dropped since Marching Cubes cannot use interpolation based on TSDF values anymore
- ▶ Saved $> 90\%$ streaming bandwidth with compression $\rightarrow$ 561.5 MBit/s (TSDF) down to 17.5 MBit/s

Results:

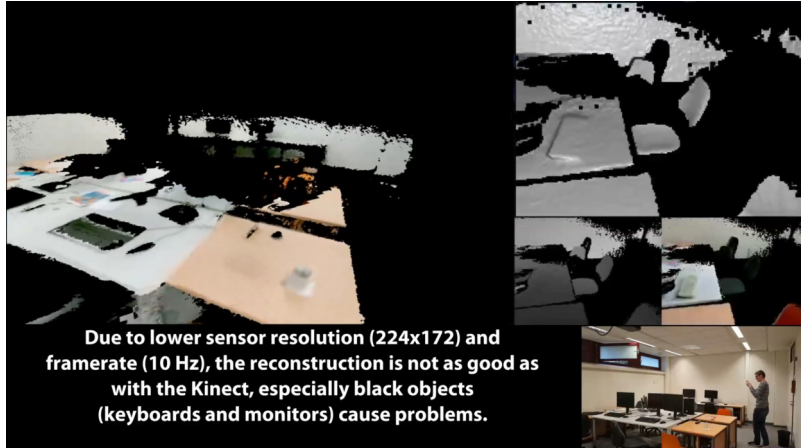


<https://youtu.be/zSPmf97Ry6I?t=80>

Results:

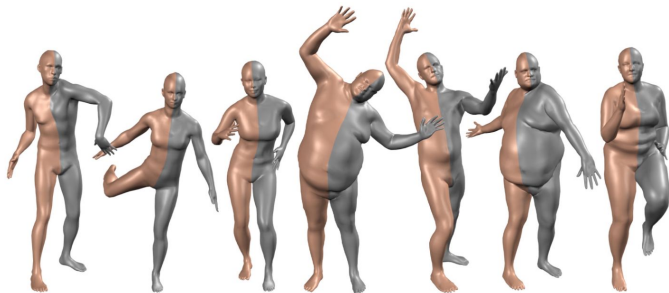


→ Also works with built-in RGB-D camera of smartphones



Next Lecture: Human Models

→ **Human Models**



- [1] B. Curless and M. Levoy. "A Volumetric Method for Building Complex Models from Range Images". In: *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '96. ACM, 1996.
- [2] A. Dai, M. Nießner, M. Zollöfer, S. Izadi, and C. Theobalt. "BundleFusion: Real-time Globally Consistent 3D Reconstruction using On-the-fly Surface Re-integration". In: *ACM Transactions on Graphics 2017 (TOG)* (2017).
- [3] S. Izadi et al. "KinectFusion: Real-time 3D Reconstruction and Interaction Using a Moving Depth Camera". In: *ACM Symposium on User Interface Software and Technology (UIST)*. 2011.
- [4] M. Levoy, K. Pulli, B. Curless, S. Rusinkiewicz, D. Koller, L. Pereira, M. Ginzton, S. Anderson, J. Davis, J. Ginsberg, et al. "The Digital Michelangelo Project: 3D Scanning of Large Statues". In: *Annual Conference on Computer Graphics and Interactive Techniques*. 2000.
- [5] R. A. Newcombe et al. "KinectFusion: Real-Time Dense Surface Mapping and Tracking". In: *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 2011.
- [6] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger. "Real-time 3D Reconstruction at Scale using Voxel Hashing". In: *ACM Transactions on Graphics (TOG)* (2013).

- [7] P. Stotko, S. Krumpen, M. B. Hullin, M. Weinmann, and R. Klein. “SLAMCast: Large-Scale, Real-Time 3D Reconstruction and Streaming for Immersive Multi-Client Live Telepresence”. In: *IEEE Transactions on Visualization and Computer Graphics* 25.5 (2019).